DATA STRUCTURE

Assignment_1

Prepared by:

Name of Student : Chaitanya Dalvi

Roll No: 19

Batch: 2023-27

Dept. of CSE

Que1. Explain sliding window protocol in detail with the help of a timing diagram?

Que 2. Difference between the Go-Back-N ARQ and Selective Repeat ARQ?

Que 3. Write a program to implement Kadane's Algorithm?

Que 4. What is a prefix sum algorithm?

Que 5. Explain pointer technique?

Que 6. Draw a timing diagram to explain Stop and Wait protocol?

Que 7. Find maximum subarray sum for the following elements:
Input: nums = [5,4,-1,7,8]
Input: nums = [1]
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]

1. Explain sliding window protocol in detail with the help of a timing diagram?

Ans:-

1. Sliding Window Protocol:

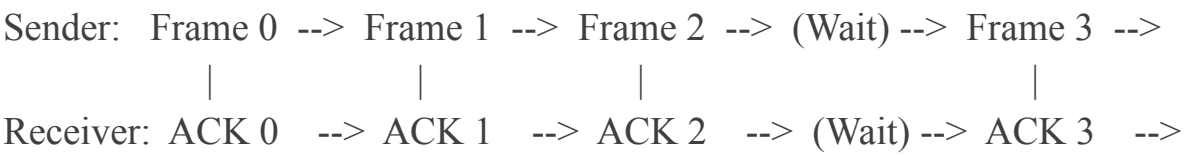
The **Sliding Window Protocol** is used in networking to manage the flow of frames between sender and receiver, ensuring data is transferred efficiently and accurately. It is a type of flow control where both sender and receiver have a "window" of frames they can send/receive without waiting for an acknowledgment for every single frame.

Key Concepts:

- **Window Size**: The number of frames that can be sent without waiting for an acknowledgment.
- **Sender Window**: Keeps track of frames sent but not yet acknowledged.
- **Receiver Window**: Tracks frames received and those yet to be acknowledged.
- **Acknowledgment (ACK)**: The receiver sends an acknowledgment back for received frames.

Working of Sliding Window Protocol:

1. The sender sends multiple frames (up to the size of the window) before receiving an acknowledgment.
2. Once the acknowledgment is received, the sender's window moves forward, allowing it to send the next set of frames.
3. If the acknowledgment for a frame is lost, the sender will eventually time out and resend that frame.



Frames 0, 1, and 2 are sent one after the other.
After receiving ACK 0, the window slides forward, allowing the sender to send Frame 3, and so on.

2. Difference between the Go-Back-N ARQ and Selective Repeat.

Ans:-

In Go-Back-N ARQ, when an error occurs, the sender retransmits all frames from the erroneous one onward, and the receiver can't buffer out-of-order frames. It uses cumulative acknowledgments, where one ACK confirms all previous frames, but this results in lower efficiency due to multiple frame retransmissions.

In Selective Repeat ARQ, only the erroneous frame is retransmitted, and the receiver can buffer out-of-order frames. It sends individual ACKs for each frame, leading to higher efficiency since only specific frames with errors are resent.

3. Write a program to implement Kadane's Algorithm?

Ans:-

```
#include <iostream>
#include <vector>
using namespace std;

int kadane_algorithm(vector<int>& nums) {
    int max_current = nums[0];
    int max_global = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        max_current = max(nums[i], max_current + nums[i]);
        if (max_current > max_global) {
            max_global = max_current;
        }
    }
    return max_global;
}

int main() {
    vector<int> nums = {5, 4, -1, 7, 8};
    cout << "Maximum Subarray Sum: " << kadane_algorithm(nums)
<< endl;
    return 0;
}
```

Example Input and Output:

- Input: `nums = [5, 4, -1, 7, 8]`
- Output: `Maximum Subarray Sum: 23`

4. What is a prefix sum algorithm?

Ans:-

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> prefix_sum(const vector<int>& nums) {
    vector<int> prefix(nums.size());
    prefix[0] = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        prefix[i] = prefix[i - 1] + nums[i];
    }
    return prefix;
}

int range_sum(const vector<int>& prefix, int left, int right) {
    if (left == 0) return prefix[right];
    return prefix[right] - prefix[left - 1];
}

int main() {
    vector<int> nums = {1, 2, 3, 4, 5};
    vector<int> prefix = prefix_sum(nums);

   int left = 1, right = 3;
    cout << "Sum of subarray from index " << left << " to " << right <<
" is: "
         << range_sum(prefix, left, right) << endl;
    return 0;  }
```

5. Explain pointer technique?

Ans:-

The **Pointer Technique** involves using pointers (or indices) to navigate through arrays or lists efficiently. There are different types of pointer techniques:

- **Two-pointer Technique**: Often used to solve problems that require searching for pairs or subarrays. For example, in a sorted array, two pointers (one starting at the beginning and one at the end) can move toward each other to find a pair that sums to a target.
  Example: Finding two elements that sum to a target in a sorted array.
- **Fast and Slow Pointer**: Used in linked lists to detect cycles or find the middle of the list. One pointer moves faster (two steps at a time) while the other moves slower (one step at a time).

6. Draw a timing diagram to explain Stop and Wait protocol?

Ans:-

The **Stop-and-Wait Protocol** is a basic flow control protocol where the sender sends one frame and waits for its acknowledgment before sending the next frame. This is inefficient for long distances or large data, but it ensures reliable delivery.

Timing Diagram:

Sender: Frame 0 --> (Wait) --> ACK 0 --> Frame 1 --> (Wait) --> ACK 1 —>

Receiver: ACK 0 --> (Wait) --> Frame 0 --> ACK 1 --> (Wait) --> Frame 1 -->

The sender waits for the acknowledgment (ACK) of Frame 0 before sending Frame 1.

7.Find maximum subarray sum for the following elements:
Input: nums = [5,4,-1,7,8]
Input: nums = [1]
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Ans:-

```
#include <iostream>
#include <vector>
using namespace std;

int kadane_algorithm(const vector<int>& nums) {
    int max_current = nums[0];
    int max_global = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        max_current = max(nums[i], max_current + nums[i]);
        if (max_current > max_global) {
            max_global = max_current;
        }
    }
    return max_global;
}

int main() {
    vector<int> nums1 = {5, 4, -1, 7, 8};
    vector<int> nums2 = {1};
    vector<int> nums3 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};

    cout << "Maximum Subarray Sum for nums1: " <<
kadane_algorithm(nums1) << endl;
```

```
    cout << "Maximum Subarray Sum for nums2: " <<
kadane_algorithm(nums2) << endl;
    cout << "Maximum Subarray Sum for nums3: " <<
kadane_algorithm(nums3) << endl;

    return 0;
}
```

Example Input and Output:

- **Input 1**: `nums = [5, 4, -1, 7, 8]`
  - Output: `Maximum Subarray Sum for nums1: 23`
- **Input 2**: `nums = [1]`
  - Output: `Maximum Subarray Sum for nums2: 1`
- **Input 3**: `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`
  - Output: `Maximum Subarray Sum for nums3: 6`